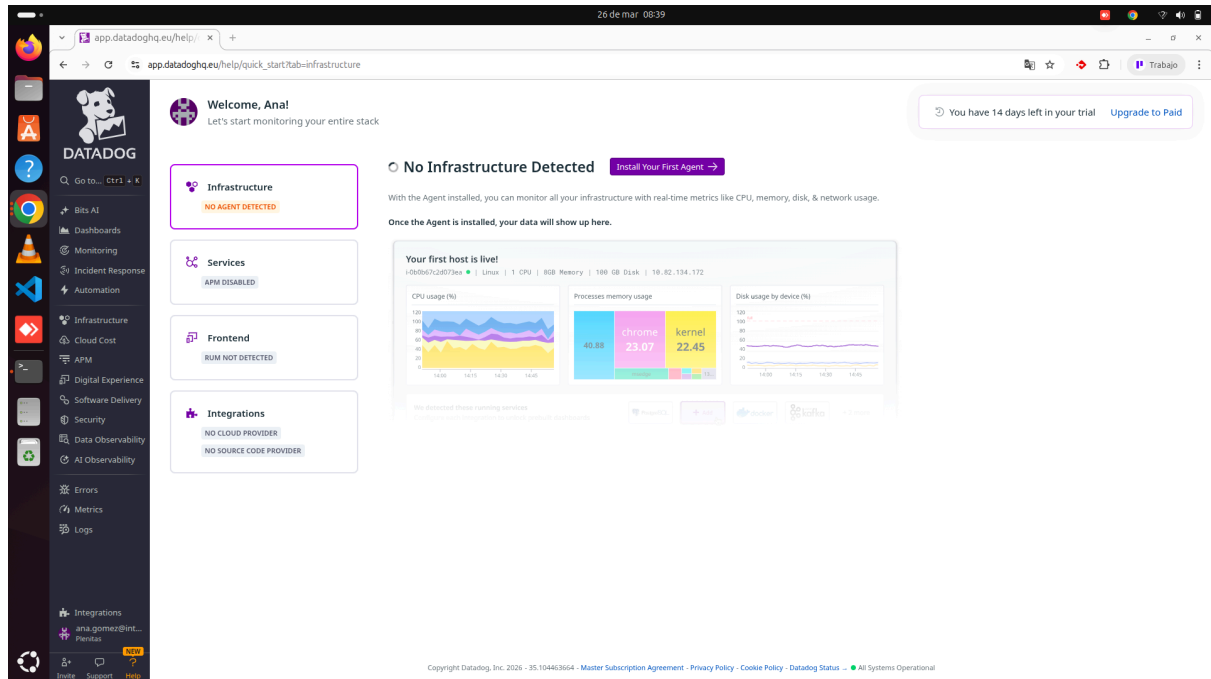


Implementación de datalog

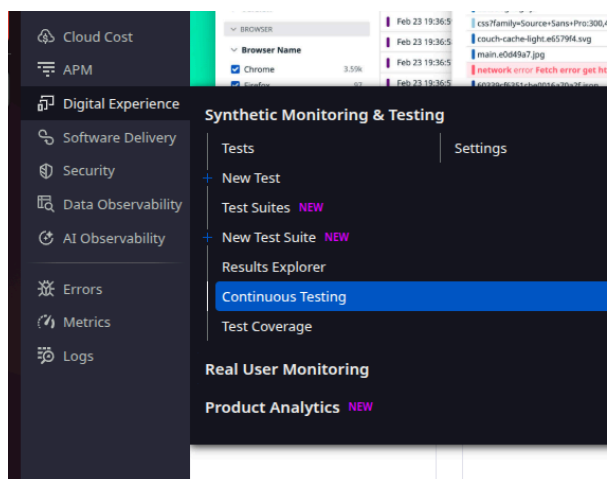
1. Iniciar sesión <https://docs.datadoghq.com/es/integrations/rum-flutter/>



Crear aplicación RUM

Digital Experience → RUM → Applications

Add Application → Flutter



Después le doy a Add Application - Flutter
Y seguir los pasos del enlace

<https://app.datadoghq.eu/rum/application/828ca0b8-8371-46e1-bba1-de4c8ca5e4f8/manage?fromCreation=true>

1

Install Dependencies

Declare the `datadog_flutter_plugin` in your `pubspec.yaml` file.

pubspec.yaml

```
1 dependencies:
2   datadog_flutter_plugin: x.x.x
```

2

Initialize RUM

Initialize the SDK with application context and start sending data

main.dart

```
1 final configuration = DatadogConfiguration(
2   clientToken: 'pub95c970e7bf49c8c35cdfdb2d5f78309',
3   env: '<ENV-NAME>',
4   site: DatadogSite.us1,
5   nativeCrashReportEnabled: true,
6   loggingConfiguration: DatadogLoggingConfiguration(),
7   rumConfiguration: DatadogRumConfiguration(
8     applicationId: '828ca0b8-8371-46e1-bba1-de4c8ca5e4f8',
9   ),
10 );
11 await DatadogSdk.runApp(configuration, TrackingConsent.granted, () async {
12   runApp(MyApp());
13 });
```

2. Instalar dependencia:
`datadog_flutter_plugin`

Tener instalado: Ninja y Cmake

3. Obtener las credenciales

Hay que coger los códigos de

- clientToken
- applicationId

Y ponerlos dentro del main

4. Configurar main.dart

```

Run | Debug | Profile
void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  final configuration = DatadogConfiguration(
    clientToken: 'pub95c970e7bf49c8c35cdfddb2d5f78309',
    env: 'dev',
    site: DatadogSite.eu1,
    nativeCrashReportEnabled: true,
    loggingConfiguration: DatadogLoggingConfiguration(),
    rumConfiguration: DatadogRumConfiguration(
      applicationId: '828ca0b8-8371-46e1-bba1-de4c8ca5e4f8',
      sessionSamplingRate: 100,
    ), // DatadogRumConfiguration
  ); // DatadogConfiguration

  await Environment.initEnvironment();

  await DatadogSdk.runApp(
    configuration,
    TrackingConsent.granted,
  ) async {
    runApp(
      const ProviderScope(
        child: MainApp(),
      ), // ProviderScope
    );
  },
);
}

```

5. Ahora arrancar la aplicación con un emulador (importante tiene que ser Android) y entrar con inicio de sesión.
6. Eventos implementados
 - Views

- login

```

@override
void initState() {
  super.initState();
  DatadogSdk.instance.rum?.startView("login", "LoginScreen");
}

@override
void dispose() {
  DatadogSdk.instance.rum?.stopView("login");
  super.dispose();
}

```

- register

```

@override
void initState() {
  super.initState();
  DatadogSdk.instance.rum?.startView("register", "RegisterScreen");
}

@override
void dispose() {
  DatadogSdk.instance.rum?.stopView("register");
  super.dispose();
}

```

- check-auth

```
@override
void initState() {
  super.initState();
  DatadogSdk.instance.rum?.startView("check-auth", "CheckAuthStatusScreen");
}

@override
void dispose() {
  DatadogSdk.instance.rum?.stopView("check-auth");
  super.dispose();
}
```

- Actions

- login-button

```
DatadogSdk.instance.rum?.addAction([
  RumActionType.tap,
  "login_button",
]);
```

- Errors

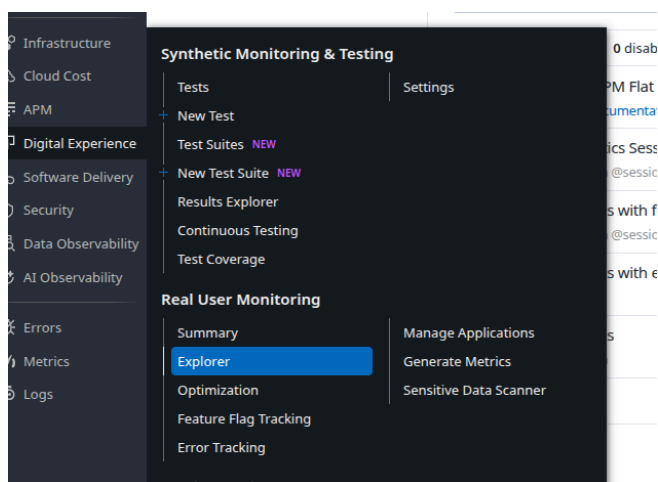
- Errores de autenticación que se capturan con:

```
DatadogSdk.instance.rum?.addErrorInfo(
  next.errorMessage,
  RumErrorSource.source,
);
```

7. Visualización en Datalog

Entrar en RUM < Explore

https://app.datadoghq.eu/rum/application/828ca0b8-8371-46e1-bba1-de4c8ca5e4f8/retention-filters?fromUser=false&from_ts=1773915768030&to_ts=1774520568030&live=true



Se pueden visualizar:

- Views (pantallas)
- Actions (interacciones)
- Errors (errores)

The screenshot shows the Datadog Flutter dashboard. At the top, there's a header with 'Welcome, Ana!' and a 'Get Started' button. Below that, the 'Application Management' section shows the 'Flutter' application, created by 'Ana Gómez Jurado' on 'Mar 26 2026'. A green banner indicates 'Instrumented successfully — your sessions are now being captured and sent to Datadog'. The left sidebar contains 'DATA COLLECTION SETTINGS' (SDK Configuration, User Data Collection, Ownership, Configuration Checklist) and 'PRODUCT SETTINGS' (Real User Monitoring, Custom Metrics, Retention Filters, Error Tracking, Debug Symbols, Product Analytics). The main content area is titled 'Retention Filters' and includes a search bar and a table of filters. The table has columns for 'FILTERS', 'RATE', 'RETAINED SESSIONS', and 'LAST MODIFIED'. It lists five filters: 'RUM-APM Flat Sampling', 'Synthetics Sessions', 'Sessions with forced replays', 'Sessions with errors', and 'Sessions'. The 'Sessions with errors' and 'Sessions' filters have toggle switches on the right.

FILTERS	RATE	RETAINED SESSIONS	LAST MODIFIED
> RUM-APM Flat Sampling View Documentation	—	—	—
> Synthetics Sessions Session @session.type: synthetics	100%	—	Mar 26, 8:42 am
> Sessions with forced replays Session @session.has_forced_replay: true	100%	—	Mar 26, 8:42 am
> Sessions with errors Error	100%	—	Mar 26, 8:42 am
> Sessions Session	100%	1.0	Mar 26, 8:42 am